L'Architecture Client-Serveur et le Protocole HTTP

Thème principal : Fonctionnement du Web, protocole HTTP, requêtes/réponses, méthodes GET/POST, sessions et codes de statut

Synthèse exécutive : Ce cours détaille les fondements du Web à travers le modèle d'interaction Client-Serveur et le protocole HTTP (HyperText Transfer Protocol). Il explique la nature nativement 'stateless' (sans état) de HTTP et les mécanismes de contournement (cookies et sessions) permettant de maintenir du contexte. Le cours analyse la structure textuelle des messages HTTP (ligne de commande/statut, en-têtes headers, ligne vide CRLF et corps body), établit un comparatif rigoureux entre les méthodes GET et POST sur les plans fonctionnel et sécuritaire, et classifie les codes de statut HTTP (2xx, 3xx, 4xx, 5xx) indispensables au débogage d'applications web.

1. L'Architecture Client-Serveur et la Nature Stateless de HTTP

L'architecture client-serveur régit l'ensemble des échanges sur le réseau Web. Contrairement au modèle Pair-à-Pair (P2P) où chaque nœud est équivalent, ce modèle repose sur une séparation stricte des rôles : le client (navigateur comme Firefox ou Chrome) émet des requêtes HTTP à destination d'un serveur applicatif (Apache, Nginx, IIS). Le serveur traite la requête en accédant aux fichiers ou à une base de données, puis retourne une réponse. Le protocole HTTP est fondamentalement 'stateless' (sans état) : une fois la réponse transmise, le serveur ferme la connexion et 'oublie' totalement l'identité du client et ses actions. Si ce fonctionnement garantit la scalabilité du serveur (qui n'a pas à maintenir en mémoire vive l'état de millions d'utilisateurs), il empêche le fonctionnement d'applications interactives (panier d'achat, espace membre). Pour surmonter cette contrainte, le serveur génère un identifiant unique qu'il transmet au client via l'en-tête 'Set-Cookie'. Le navigateur stocke ce cookie et le réémet à chaque requête ultérieure, permettant de reconstituer le contexte utilisateur via les sessions.

- Asymétrie Client-Serveur : le client (navigateur) est l'initiateur actif de la demande tandis que le serveur écoute de manière passive sur un port dédié (80/443).
- Protocole Stateless (Sans état) : chaque transaction requête-réponse est totalement amnésique et indépendante des échanges précédents.
- Gestion de Session et Cookies : utilisation de l'en-tête Set-Cookie pour injecter un identifiant de session (Session ID) et simuler une persistance d'état.

Cas concret enseignant : *Le professeur compare HTTP à un guichetier amnésique : à chaque fois que vous vous présentez à son guichet, même deux secondes après la précédente interaction, il vous traite comme un inconnu. Pour qu'il vous reconnaisse, vous devez lui présenter votre badge (le cookie de session) à chaque échange. De plus, lors de l'accès au site de l'IUT, le navigateur n'envoie pas une seule requête mais plusieurs dizaines de requêtes HTTP simultanées pour récupérer la page HTML, puis les feuilles CSS, les scripts JS et les images.*

2. Structure des Messages HTTP et Comparatif GET vs POST

Chaque message HTTP (requête ou réponse) est structuré sous forme d'un bloc de texte brut composé de quatre sections : 1. La ligne initiale : ligne de commande pour une requête (ex: 'GET /index.html HTTP/1.1') ou ligne de statut pour une réponse (ex: 'HTTP/1.1 200 OK'). 2. Les en-têtes (Headers) : paires 'Clé: Valeur' fournissant le contexte. L'en-tête 'Host' est obligatoire en HTTP/1.1 pour permettre l'hébergement virtuel de plusieurs sites sur une même IP. 3. Une ligne vide (CRLF) : marque la séparation syntaxique obligatoire avant le corps. 4. Le corps (Body) : contient les données transmises (HTML, JSON, données de formulaire). Le choix entre GET et POST dépend de l'intention : - GET est idempotente (sa répétition ne modifie pas l'état du serveur) et place ses données dans l'URL. Elle ne doit servir qu'à la lecture. - POST modifie l'état du serveur et transmet ses données dans le body, ce qui évite de polluer l'URL et d'exposer des données sensibles dans l'historique.

- Format Textuel Normalisé : structure en 4 parties (ligne initiale, en-têtes headers, ligne vide CRLF, corps body optionnel).

• En-têtes (Headers) : métadonnées sous forme Clé: Valeur décrivant l'hôte (Host), le client (User-Agent) ou le type de contenu (Content-Type).

• Méthode GET : conçue pour la lecture/consultation, idempotente et sûre, véhiculant ses paramètres dans la Query String de l'URL (limite ~2048 caractères).

• Méthode POST : conçue pour la création/modification de données sur le serveur, encapsulant ses variables dans le corps (body) sans restriction de taille.

Cas concret enseignant : *Le professeur insiste sur la vulnérabilité liée à l'envoi d'un mot de passe via GET : les données apparaissant en clair dans l'URL de la barre d'adresse, elles s'enregistrent dans l'historique du navigateur et dans les fichiers de logs du serveur. C'est la porte ouverte au 'shoulder surfing' (regard par-dessus l'épaule) et aux fuites de sécurité.*

3. Classification et Décodage des Codes de Statut HTTP

Le serveur renvoie un code numérique à trois chiffres dans la ligne de statut de sa réponse afin de communiquer le résultat de la requête sans nécessiter l'analyse du corps. Les classes de codes sont réparties ainsi : - 2xx : Succès du traitement. - 3xx : Redirection du client vers une autre URI ou exploitation du cache. - 4xx : Erreur imputable au client, par exemple lorsque l'URL demandée n'existe pas (404) ou que l'accès est refusé faute de droits (403). - 5xx : Erreur imputable au serveur. La requête du client était valide, mais le code applicatif côté serveur a planté (500) ou le service est indisponible (503).

• Codes 1xx : requêtes d'information et traitements en cours.

• Codes 2xx (Succès) : la requête a été reçue, comprise et traitée avec succès (ex: 200 OK, 201 Created).

• Codes 3xx (Redirection) : actions complémentaires requises (ex: 301 Moved Permanently, 304 Not Modified).

• Codes 4xx (Erreur Client) : anomalie sous la responsabilité du client (ex: 404 Not Found, 403 Forbidden).

• Codes 5xx (Erreur Serveur) : défaillance interne du serveur sur une requête pourtant valide (ex: 500 Internal Server Error, 503 Service Unavailable).

Cas concret enseignant : *Le professeur raconte qu'à ses débuts, un point-virgule manquant dans son script PHP entraînait un plantage de la connexion à la base de données. Le serveur renvoyait alors une erreur 500 (Internal Server Error). Les utilisateurs pensaient à tort que le site était piraté, alors qu'il s'agissait simplement d'une erreur de syntaxe dans son code backend.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : Pour un formulaire de connexion, il faut impérativement utiliser POST. Si vous utilisez GET, les identifiants et mots de passe sont ajoutés à la suite de l'URL dans la barre d'adresse (ex: login.php?user=matei&pass;=secret). Ils deviennent visibles par quiconque regarde l'écran, sont conservés dans l'historique du navigateur et enregistrés en clair dans les logs du serveur. En utilisant POST, les données sont véhiculées de manière invisible dans le corps (body) de la requête HTTP, levant la contrainte de taille et évitant l'exposition dans l'URL.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0 Identifier la méthode utilisée, la ressource sollicitée, l'hôte cible, le rôle de l'en-tête User-Agent, et déterminer le code de statut retourné par le serveur si la ressource demandée n'existe pas.

Résolution pas à pas :
1. Ligne de requête ('GET /images/logo.png HTTP/1.1') : - Méthode : GET (demande de lecture/récupération d'une ressource sans modification de l'état du serveur). - URI : '/images/logo.png' (chemin d'accès absolu vers l'image). - Version : HTTP/1.1. 2. En-tête Host ('Host: iutbayonne.univ-pau.fr') : - Spécifie le nom de domaine visé. Cet en-tête est obligatoire en HTTP/1.1 pour permettre l'hébergement virtuel (plusieurs sites sur la même adresse IP). 3. En-tête User-Agent ('User-Agent: Mozilla/5.0') : - Identifie le logiciel client (navigateur, version, système

d'exploitation) qui émet la requête. 4. Diagnostic d'absence de la ressource : - Si le fichier 'logo.png' est absent du dossier '/images/' sur le serveur, l'URL saisie par le client cible un élément inexistant. - Il s'agit d'une erreur de responsabilité client (classe 4xx). - Le serveur retournera le code de statut '404 Not Found'.

Résultat : Méthode : GET | Ressource : /images/logo.png | Hôte : iutbayonne.univ-pau.fr | Client : Mozilla/5.0 | Code en cas d'absence : 404 Not Found (Erreur Client).

■ Auto-Évaluation Flash d'Examen

Q : Pourquoi qualifie-t-on le protocole HTTP de 'stateless' (sans état) et quelle est la conséquence sur la gestion des sessions ?

R : HTTP est dit 'stateless' car le serveur oublie l'identité du client et le contexte dès la réponse envoyée. Pour gérer une session (ex: connexion utilisateur), on doit associer un cookie contenant un Session ID envoyé automatiquement par le navigateur à chaque requête.

Q : Quelle est la différence fondamentale de responsabilité entre un code de statut HTTP 4xx et un code 5xx ?

R : La classe 4xx signale une erreur sous la responsabilité du client (ex: mauvaise URL avec 404 ou absence de droits avec 403), tandis que la classe 5xx signale un dysfonctionnement interne au serveur (ex: crash de script ou panne de base de données avec 500) alors que la requête était valide.

Q : Pourquoi la méthode HTTP GET ne doit-elle jamais être employée pour transmettre un mot de passe ou une donnée sensible ?

R : Parce que GET transmet ses paramètres directement dans l'URL. Les données sensibles apparaissent ainsi en clair dans la barre d'adresse, sont enregistrées dans l'historique du navigateur et figurent dans les fichiers de logs du serveur.